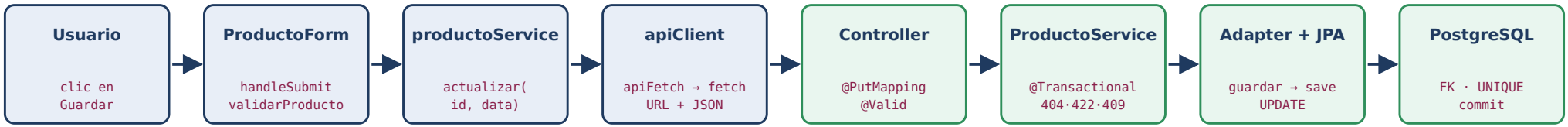
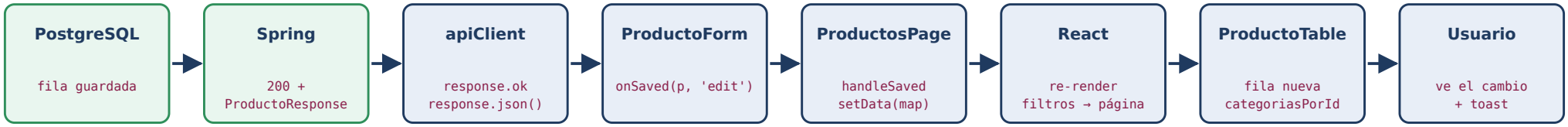
Flujo fullstack de StockFlow, dibujado

Programación Aplicada 2026-2 · 2do parcial · complemento de `respuestas-2do-parcial.pdf` (preguntas 21, 25, 50 y desafío 1). El examen pide «dibujar el flujo completo: usuario → React → hook/service → HTTP → Spring Boot → respuesta → estado → re-render».

Versión para la pizarra: editar un producto (PUT), ida



Vuelta



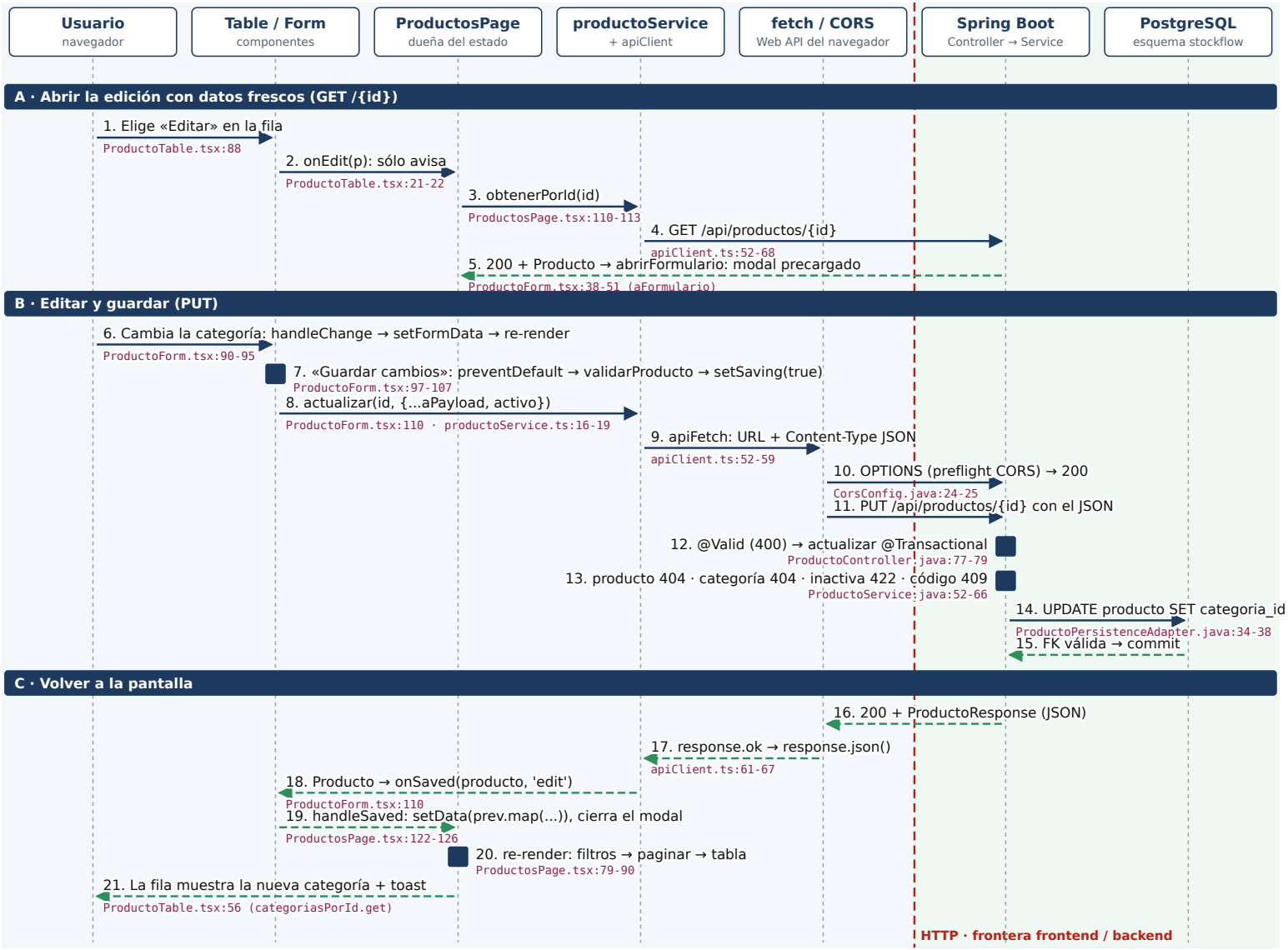
Azul: frontend (corre en el navegador) · **Verde**: backend ya implementado en el parcial 1 · la frontera entre ambos es la petición HTTP.

Cómo usarlo en la defensa

- Dibuja primero estas dos filas (son 16 cajas) y marca la frontera HTTP. Después agrega el detalle de la página 2 sólo si te lo piden.
- Por cada caja, nombra la pieza real (archivo y función) y qué decide: «ProductoForm valida y arma el payload», «el service conoce la ruta», «apiClient revisa response.ok».
- Para la vuelta, el punto clave es que React re-renderiza porque se llamó a un setter con un array nuevo (`setData(prev.map(...))`), no «porque llegó la respuesta».
- La página 4 responde «¿dónde se rompería?»: cada falla apunta al número de paso de la página 2.

Detalle: editar un producto y cambiarle la categoría (PUT)

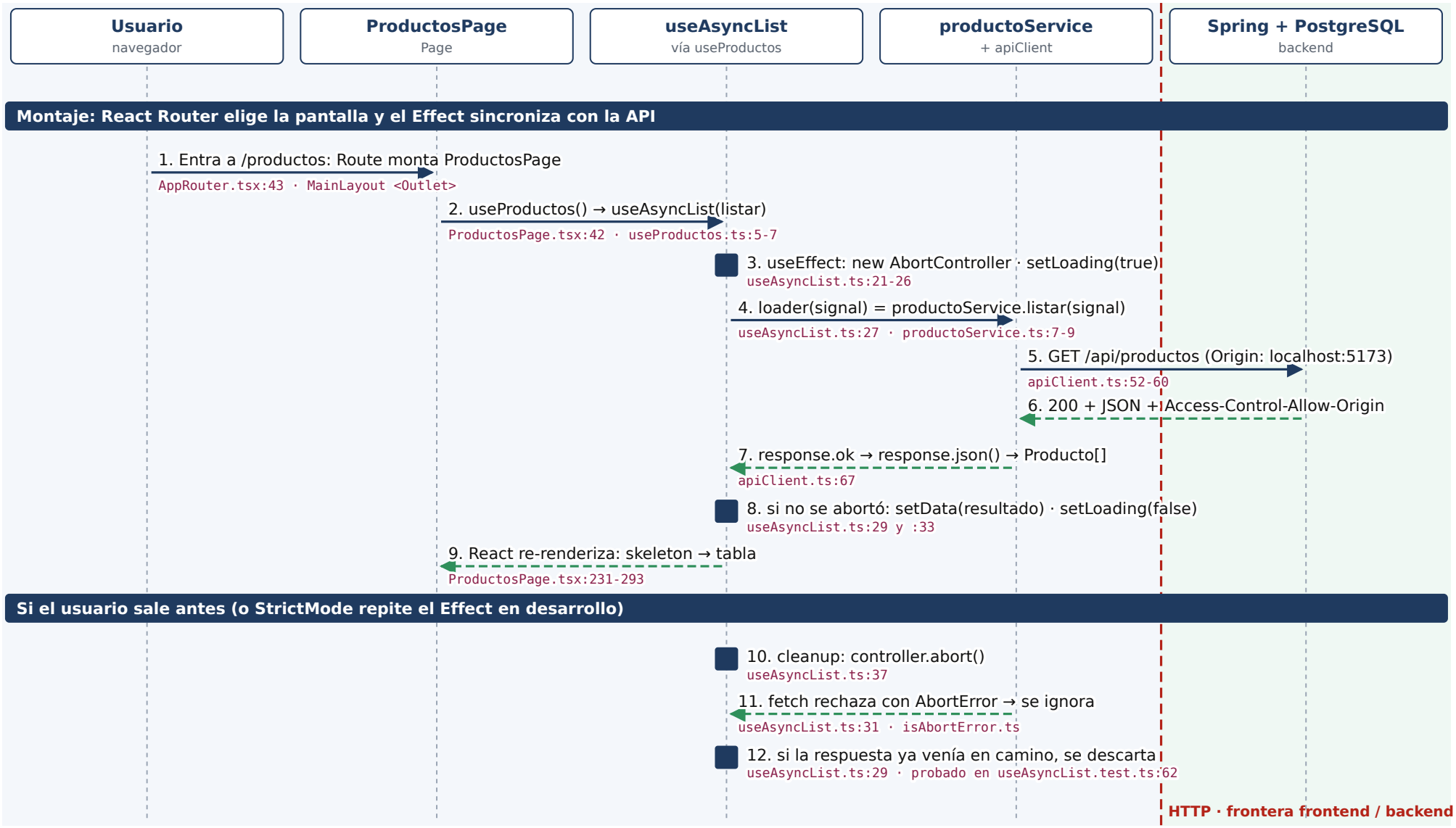
Es la operación que sugiere la pregunta 50: toca formulario, relación 1:N, PUT, reglas de negocio y actualización de la tabla.



Flecha llena: llamada o petición · **flecha punteada verde**: respuesta · cuadrado azul: algo que pasa dentro de ese carril · **línea roja**: frontera HTTP. Las rutas de frontend cuelgan de `webstorm/stockflow-frontend/src/` y las de backend de `intellij/stockflow-backend/src/main/java/com/stockflow/`.

Detalle: cargar la lista al entrar a /productos (GET)

Preguntas 21, 38 y 39: el Effect sincroniza la pantalla con la API y su cleanup cancela lo que ya no sirve.



En desarrollo, StrictMode ejecuta setup → cleanup → setup: en Network se ve un primer GET *(canceled)* y un segundo que completa. En producción hay uno solo.

Dónde se rompería el flujo

Pregunta 50 («diagnosticar dónde se rompería ante URL incorrecta, CORS, payload inválido, 404, 409, 500 o respuesta con forma inesperada») y desafío 3. La columna «Paso» apunta al diagrama del PUT.

Falla	Paso	Qué se ve en Network / Console	Qué capa lo detecta	Qué ve el usuario
URL mal escrita dentro de /api	**4 / 11**	404 en /api/producto	Backend (sin mapping) → ApiError	Alert con el mensaje del backend + Reintentar
Prefijo o puerto incorrecto	**4 / 11**	«CORS error» o ERR_CONNECTION_REFUSED	Navegador → TypeError	«No se pudo conectar con el backend…»
Origen no permitido (CORS)	**10**	OPTIONS sin Access-Control-Allow-Origin; Console «blocked by CORS policy»	Navegador → TypeError	«No se pudo conectar con el backend…»
Formulario inválido	**7**	Ninguna petición	validarProducto (UI)	Error junto al campo y foco en él
Payload inválido para el backend	**12**	400 con fieldErrors	Bean Validation (@Valid)	Errores por campo en el formulario
Producto o categoría inexistente	**13**	404	ProductoService	«El registro ya no existe» + reload()
Mover a una categoría inactiva	**13**	422	Regla de negocio	«La operación no cumple una regla del inventario»
Código repetido	**13 / 14**	409	ProductoService o UNIQUE en PostgreSQL	Mensaje en el modal; el formulario se conserva
Error interno	**12-15**	500	GlobalExceptionHandler	«El servidor tuvo un problema»; no se finge éxito
Respuesta con forma inesperada	**17**	200, pero el JSON no coincide con el modelo	Nadie en runtime (TS no existe al ejecutar)	Celdas vacías; si el render se rompe, AppErrorBoundary
201/200 en Network pero la tabla no cambia	**18-20**	La respuesta está bien	Estado del frontend	Revisar onSaved, setData con array nuevo, filtros y página

Método: abre Network antes de reproducir. Si la petición no aparece, el problema está antes del paso 9 (validación o JavaScript). Si aparece, mira status, Payload y Response, y compáralos con el contrato probado en Swagger o Postman.